# LAB WORK – PROGRAMMABLE DATA PLANE AND P4

## 1. INTRODUCTION

### 1.1 MOTIVATION

In this lab work, structured in a tutorial/walkthrough manner, we will dive into the concept of programmable data plane technology. For such journey we will explore the P4 programming language. This lab is divided into two parts: in the first part we will follow a step-by-step guide prepared by the P4 developers to understand the basics of P4 and programmable data planes, and in the second part we will develop a specific P4 application to identify and mark a set of packets.

A programmable data plane is a concept in networking where the data plane - the part of a network device like a router or switch responsible for forwarding packets - can be customized or programmed to perform specific tasks based on the network's needs. Traditionally, the data plane's functionality is fixed and determined by the hardware manufacturer, limiting flexibility. However, a programmable data plane allows network operators to define how packets are processed, enabling more control over network behavior. Such flexibility enables networks to be more adaptable, allowing for custom packet processing, the implementation of new protocols, and advanced features without the need for new hardware.

Programmability in the data plane is typically achieved through specialized programming languages or application programming interfaces (APIs). One of the most well-known technologies enabling programmable data planes is the P4 language, which allows developers to specify how packets are parsed, processed, and forwarded. Instead of relying on fixed-function pipelines, which are pre-determined paths for packet processing in traditional hardware, P4, which stands for Programming Protocol-independent Packet Processors, allows for custom rules and processing logic. The language is declarative, meaning that developers specify what they want the network device to do, rather than providing step-by-step instructions. The P4 compiler then maps this behavior onto the hardware, optimizing it for the specific platform.

Besides P4 programming language, and the already explored *Mininet* environment, in this lab we will also make use of two other technologies: the BMv2 and the P4runtime. BMv2 (Behavioral Model version 2) is a widely-used software switch that serves as a reference implementation for the P4 programming language. It is primarily used for testing, experimentation, and development in environments where physical P4-compatible hardware is not available or practical to use. BMv2 allows developers to simulate how their P4 programs will behave on actual network devices. It operates as a virtual switch, compiling P4 programs and creating a virtual network device that can process packets according to the rules defined in the P4 code.

On the other hand, P4runtime provides a standardized way to control and manage P4-programmable network devices at runtime, i.e., it allows for the dynamic configuration of devices that have been programmed using the P4 language. This means that we can modify the behavior of a network device on the fly, without needing to recompile or redeploy the P4 program that defines the device's packet processing logic.

### 1.2 LEARNING OUTCOMES

At the end of this lab, you should know:

- The basic concepts about programming data planes;
- To prepare virtualized network environments, through *Mininet* environment, to apply and evaluate network rules;
- The purpose and structure of P4 programming language.

## 2. PREPARATION

For this lab we will follow the P4 tutorials prepared by the P4 developers, which can be found here: https://github.com/p4lang/tutorials

**It is recommended that you prepare the lab environment before the first lab class.** For that please refer to *Obtaining required software* section. There are two VM images available with different Ubuntu versions, 20.04 and 24.04, but you should stick with the oldest one – no problem with that.

A P4 tutorial presentation, prepared by the P4 developers, is also available in the repository: you should also read it before starting the lab work.

## 3. TASKS

This lab consists in 2 tasks. First we will understand the basics of P4 programming language through the implementation of *simple* IPv4 forwarding. The idea is to you develop logic for updating MAC addresses, decrementing TTL values, and forwarding packets based on predefined rules. In the second task we will explore P4 to mark and filter IPv4 according to a set of rules.

### 3.1 BASIC FORWARDING

In this task you should follow the Basic forwarding exercise provided by the P4 developers, available here: https://github.com/p4lang/tutorials/tree/master/exercises/basic

The goal is to write a P4 program that implements basic forwarding. The switch must perform the following actions for every packet: (i) update the source and destination MAC addresses, (ii) decrement the time-to-live (TTL) in the IP header, and (iii) forward the packet out the appropriate port.

In the following there is a list of the most important steps to complete this task. For a complete walkthrough one should check the repository's main page, where you will also find some troubleshooting.

1. Introduction
   a. Your switch will have a single table, which the control plane will populate with static rules. Each rule will map an IP address to the MAC address and output port for the next hop. We have already defined the control plane rules, so you only need to implement the data plane logic of your P4 program.

b. We will use the following topology for this exercise. It is a single pod of a fat-tree topology and henceforth referred to as pod-topo, as shown in Figure 1.

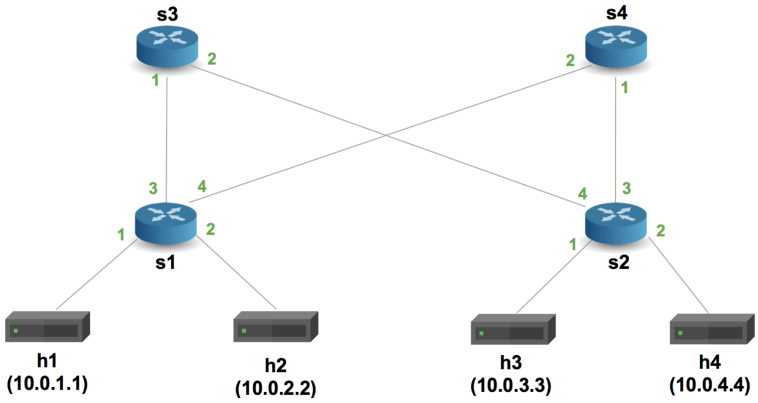



Figure 1 – Network topology to be used in this lab.

c. Our P4 program will be written for the V1Model architecture implemented on P4 bmv2 software switch. The architecture file for the V1Model can be found at: /usr/local/share/p4c/p4include/v1model.p4. This file describes the interfaces of the P4 programmable elements in the architecture, the supported externs, as well as the architecture's standard metadata fields. You should look at it.

2. Run the (incomplete) starter code
   a. As mentioned before, a basic P4 program is available. Let us compile it and bring up a switch in *Mininet* to test its behavior.

   b. In your shell, run `make run`. This will:
      i. compile `basic.p4`, and
      ii. start the pod-topo in *Mininet* and configure all switches with the appropriate P4 program + table entries, and
      iii. configure all hosts with the commands listed in pod-topo/topology.json

   c. You should now see a Mininet command prompt. Try to ping between hosts in the topology:
      i. `mininet> h1 ping h2`
      ii. `mininet> pingall`

   d. Type exit to leave each xterm and the *Mininet* command line. Then, to stop *Mininet* `make stop`, and to delete all pcaps, build files, and logs `make clean`

   e. The ping failed because each switch is programmed according to `basic.p4`, which drops all packets on arrival. Your job is to extend this file so it forwards packets.

   f. A log folder is available with logs from the several network elements. You should use it to debug your program.

3. Implement basic L3 forwarding
   a. The `basic.p4` file contains a skeleton P4 program with key pieces of logic replaced by **TODO** comments. Your implementation should follow the structure given in this file: replace each **TODO** with logic implementing the missing piece.

   b. A complete `basic.p4` will contain the following components:
      i. Header type definitions for Ethernet (`ethernet_t`) and IPv4 (`ipv4_t`).
      ii. **TODO**: Parsers for Ethernet and IPv4 that populate `ethernet_t` and `ipv4_t` fields.
      iii. An action to drop a packet, using `mark_to_drop()`.
      iv. **TODO**: An action (called `ipv4_forward`) that:
         1. Sets the egress port for the next hop.
         2. Updates the ethernet destination address with the address of the next hop.
         3. Updates the ethernet source address with the address of the switch.
         4. Decrements the TTL.
      v. **TODO**: A control that:
         1. Defines a table that will read an IPv4 destination address, and invoke either `drop` or `ipv4_forward`.
         2. An apply block that applies the table.
      vi. **TODO**: A deparser that selects the order in which fields inserted into the outgoing packet.
      vii. A package instantiation supplied with the parser, control, and deparser. (In general, a package also requires instances of checksum verification and recomputation controls. **These are not necessary for this tutorial and are replaced with instantiations of empty controls.**)

   c. After the implementation, execute the steps in stage 2, and you should be able to ping between any two hosts in the topology[1].

## 3.1 Mark and Filter

In this task we will extend the functionalities developed in the previous task to filter and mark suspicious traffic. We will use the same network topology as before. Download, from the course website, a zipper folder named `filter.zip` containing a set of files needed to complete the task, and place it inside the exercises folder created in the first task of this lab.

[1] There is a possible solution for this task available in the solution folder. You are free to use it to validate and compare to your solution. Remember that your report will have to justify all the steps you did to complete the task.

1. Introduction
   a. Packet headers
      i. Packets that enter the network, transmitted by the hosts, will have an extra header, called filter header. The header sits between the IP and UDP headers, as shown by Figure 2.

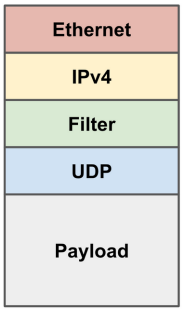



Figure 2 – Packets headers in task 2.

      ii. When the protocol filed in the IP header is 146, it means that the next header is the filter header. It has two fields:
         1. `susp` (1 byte), which should be set to one if a packet is suspicious and zero otherwise. Originally, this field is zero for all packets that enter the network.
         2. `proto` (1 byte), which determines what protocol the next header belongs to. All the packets that enter the network will have this fields set to 17, which means the next header will be a UDP header.

   b. Filter table
      i. Packets are declared suspicious based on their source IP address and source (UDP) port. There is a list of (IP, source port) pairs available in susp.txt. We want to define a table in the switch data plane that will match on source IP address and source port of the packet, and if the value matches the list, set the `susp` field in `filter` header to one.

   c. Deparser
      i. It specifies, using `emit` statements, which headers should go back in the packet and in what order. If a header is invalid (e.g., has not been parsed), the corresponding `emit` statement will not add it to the packet.

   d. Controller
      i. The controller, available in `mycontroller.py` reads the suspicious list and installs rules in the `filter` table accordingly.

2. Run the code
   a. In your shell, run `make run`. This will:
      i. compile `filter.p4`,
      ii. start a *Mininet* instance with four switches (s1, s2, s3, s4) configured as shown in the pod topology, each connected to one host (h1, h2, h3, h4), and

iii. assign IPs of 10.0.1.1, 10.0.2.2, 10.0.3.3, and 10.0.4.4 to the respective hosts.

b. You should now see a prompt that asks you to run the controller first. Open another shell and run the controller `./mycontroller.py` This will install the `filter.p4` program on the switches and push the forwarding and filter rules.

c. In the first shell, write `done` and press enter.

d. The script in the first shell will then continue running and send packets between hosts. Some of these packets will be suspicious and some will not.

e. After the packets are sent, the script will check whether they were correctly marked as suspicious or not. Any mismatches will be reported on the screen.

f. Note that while you will not see the *Mininet* prompt in this exercise, the `run_exercise.py` script is using *Mininet* behind the scenes to create the network and send traffic.

3. To complete this task you will have to:
   a. Modify the `filter.p4` program in the designated areas to define the filter and UDP headers, instantiate them, and parse them.

   b. Modify the `MyIngress` control block in the `filter.p4` program to add the `filter` table with appropriate keys and actions. Later, we will populate this table from the controller. Also extend the code in the apply block in `MyIngress` to first check if the `filter` and `udp` headers are valid, and if yes, apply the `filter` table before performing IP forwarding.

   c. Modify `MyDeparser` to add `filter` and `udp` headers back to the outgoing packet.

   d. Fill in the `writeFilterRule` function in `mycontroller.py` to populate the `filter` table.

## 4. DELIVERY

A report (max of 20 plain pages 10 ≤ letter size ≤ 12 and 1.0 ≤ spacing ≤ 1.5 lines) should be delivered in due time. If annexes are required, these must not exceed 5 pages. The report is submitted through Moodle, and is due on: **CHECK DATES ON MOODLE.**

The report should be organized in such a way that includes the following sections: cover page with proper identification, introduction, objectives, experimental methodologies, result analysis and conclusions. **Obtained results must be analysed and explained.**

You must also submit all the files that were changed/modified during this lab.